

Project 3

Reinforcement learning and Inverse Reinforcement learning

Due on Wednesday, May 20, 2026 by 11:59 PM PDT

1 Introduction

Reinforcement Learning (RL) is the task of learning from interaction to achieve a goal. The learner and the decision maker is called the **agent**. The thing it interacts with, comprising everything outside the agent, is called the **environment**. These interact continually, the agent selecting **actions** and the environment responding to those actions by presenting **rewards** and new **states**.

In the first part of the project, we will learn the optimal policy of an agent navigating in a 2-D environment. We will implement the Value iteration algorithm to learn the optimal policy.

Inverse Reinforcement Learning (IRL) is the task of extracting an expert's reward function by observing the optimal policy of the expert. In the second part of the project, we will explore the application of IRL in the context of apprenticeship learning.

2 Reinforcement learning (RL)

The two main objects in Reinforcement learning are:

- Agent
- Environment

In this project, we will learn the optimal policy of a **single agent** navigating in a 2-D environment.

2.1 Environment

In this project, we assume that the environment of the agent is modeled by a **Markov Decision Process (MDP)**. In a MDP, agents occupy a state of the environment and perform actions to change the state they are in. After taking an action, they are given some representation of the new state and some reward value associated with the new state.

An MDP formally is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}_{ss'}^a, \mathcal{R}_{ss'}^a, \gamma)$ where:

- $\mathcal{S}$ is a set of **states**
- $\mathcal{A}$ is a set of **actions**
- $\mathcal{P}_{ss'}^a$ is a set of **transition probabilities**, where $\mathcal{P}_{ss'}^a$ is the probability of transitioning from state $s \in \mathcal{S}$ to state $s' \in \mathcal{S}$ after taking action $a \in \mathcal{A}$
 - $\mathcal{P}_{ss'}^a = \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a)$
- Given any current state and action, s and a , together with any next state, s' , the expected value of the next reward is $\mathcal{R}_{ss'}^a$
 - $\mathcal{R}_{ss'}^a = \mathbb{E}(r_{t+1} | s_t = s, a_t = a, s_{t+1} = s')$
- $\gamma \in [0, 1)$ is the discount factor, and it is used to compute the present value of future reward
 - If γ is close to 1 then the future rewards are discounted less
 - If γ is close to 0 then the future rewards are discounted more

In the next few subsections, we will discuss the parameters that will be used to generate the environment for the project.

2.1.1 State space

In this project, we consider the state space to be a 2-D square grid with 100 states. The 2-D square grid along with the numbering of the states is shown in figure 1

	0	1	2	3	4	5	6	7	8	9	10
0	0	10	20	30	40	50	60	70	80	90	
1	1	11	21	31	41	51	61	71	81	91	
2	2	12	22	32	42	52	62	72	82	92	
3	3	13	23	33	43	53	63	73	83	93	
4	4	14	24	34	44	54	64	74	84	94	
5	5	15	25	35	45	55	65	75	85	95	
6	6	16	26	36	46	56	66	76	86	96	
7	7	17	27	37	47	57	67	77	87	97	
8	8	18	28	38	48	58	68	78	88	98	
9	9	19	29	39	49	59	69	79	89	99	
10											

Figure 1: 2-D square grid with state numbering

2.1.2 Action set

In this project, we consider the action set($\mathcal{A}$) to contain the 4 following actions:

- [Move Right](#)

- Move Left
- Move Up
- Move Down

The 4 types of actions are displayed in figure 2

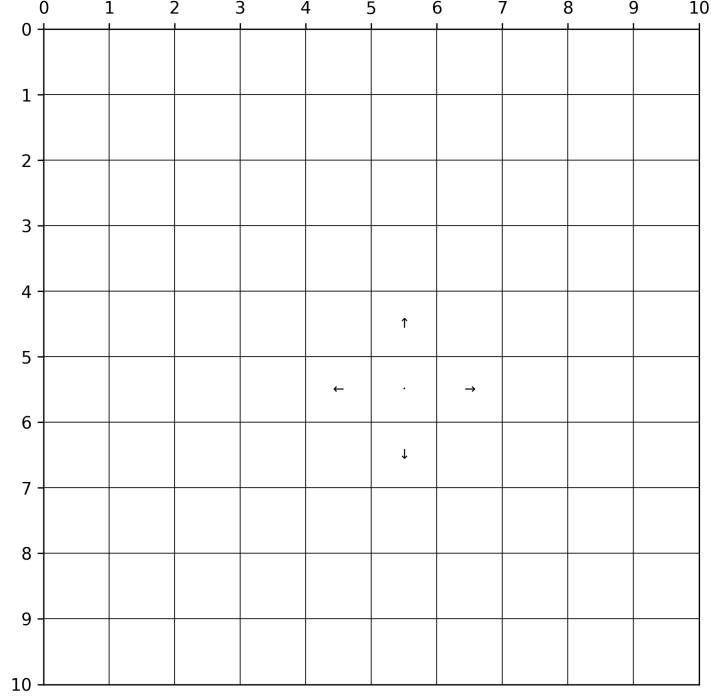


Figure 2: 4 types of action

From the above figure, we can see that the agent can take 4 actions from the state marked with a dot.

2.1.3 Transition probabilities

In this project, we define the transition probabilities in the following manner:

1. If state s' and s are not neighboring states in the 2-D grid, then

$$\mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a) = 0$$

s' and s are neighbors in the 2-D grid if you can move to s' from s by taking an action a from the action set $\mathcal{A}$. We will consider a state s to be a neighbor of itself. For example, from figure 1 we can observe that states 1 and 11 are neighbors (we can transition from 1 to 11 by moving right) but states 1 and 12 are not neighbors.

2. Each action corresponds to a movement in the intended direction with probability $1 - w$, but has a probability of w of moving in a random direction instead due to wind. To illustrate this, let's consider the states shown in figure 3

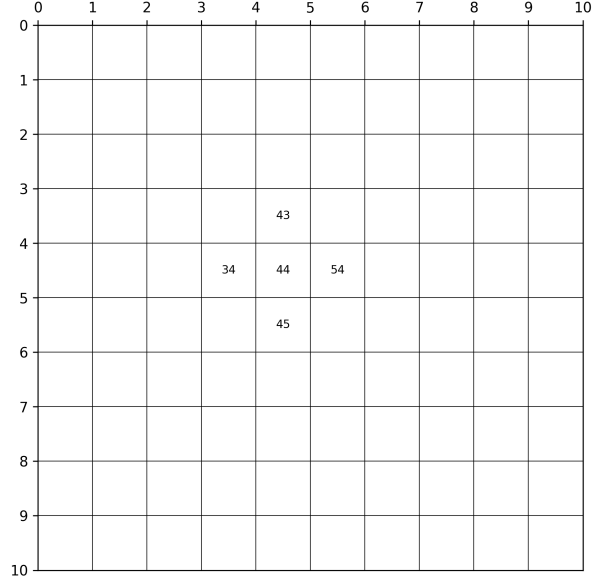


Figure 3: Inner grid states (Non-boundary states)

The transition probabilities for the non-boundary states shown in figure 3 are given below:

$$\begin{aligned}\mathbb{P}(s_{t+1} = 43 | s_t = 44, a_t = \uparrow) &= 1 - w + \frac{w}{4} \\ \mathbb{P}(s_{t+1} = 34 | s_t = 44, a_t = \uparrow) &= \frac{w}{4} \\ \mathbb{P}(s_{t+1} = 54 | s_t = 44, a_t = \uparrow) &= \frac{w}{4} \\ \mathbb{P}(s_{t+1} = 45 | s_t = 44, a_t = \uparrow) &= \frac{w}{4}\end{aligned}$$

From the above calculation it can be observed that if the agent is at a non-boundary state then it has 4 neighbors excluding itself and the probability w is uniformly distributed over the 4 neighbors. Also, if the agent is at a non-boundary state then it transitions to a new state after taking an action ($\mathbb{P}(s_{t+1} = 44 | s_t = 44, a_t = \uparrow) = 0$)

3. If the agent is at one of the four corner states (0,9,90,99), the agent stays at the current state if it takes an action to move off the grid or is blown off the grid by wind. The actions can be divided into two categories:

- Action to move off the grid
- Action to stay in the grid

To illustrate this, let's consider the states shown in figure 4

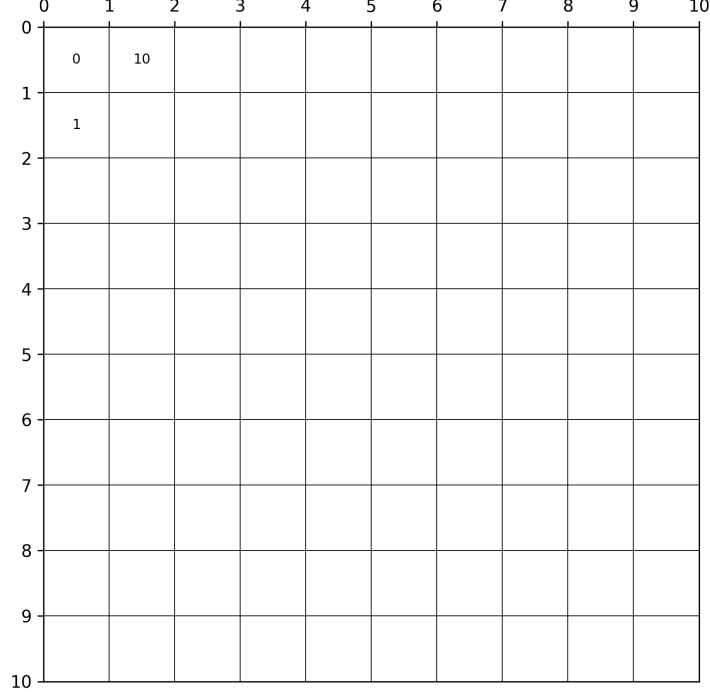


Figure 4: Corner states

The transition probabilities for taking an action to move off the grid are given below:

$$\begin{aligned}\mathbb{P}(s_{t+1} = 10 | s_t = 0, a_t = \uparrow) &= \frac{w}{4} \\ \mathbb{P}(s_{t+1} = 1 | s_t = 0, a_t = \uparrow) &= \frac{w}{4} \\ \mathbb{P}(s_{t+1} = 0 | s_t = 0, a_t = \uparrow) &= 1 - w + \frac{w}{4} + \frac{w}{4}\end{aligned}$$

The transition probabilities for taking an action to stay in the grid are given below:

$$\begin{aligned}\mathbb{P}(s_{t+1} = 10 | s_t = 0, a_t = \rightarrow) &= 1 - w + \frac{w}{4} \\ \mathbb{P}(s_{t+1} = 1 | s_t = 0, a_t = \rightarrow) &= \frac{w}{4} \\ \mathbb{P}(s_{t+1} = 0 | s_t = 0, a_t = \rightarrow) &= \frac{w}{4} + \frac{w}{4}\end{aligned}$$

At a corner state, you can be blown off the grid in two directions. As a result, we have $\mathbb{P}(s_{t+1} = 0 | s_t = 0, a_t = \rightarrow) = \frac{w}{4} + \frac{w}{4}$ since we can be blown off the grid in two directions and in both the cases we stay at the current state.

4. If the agent is at one of the edge states, the agent stays at the current state if it takes an action to move off the grid or is blown off the grid by wind. The actions can be divided into two categories:
 - Action to move off the grid
 - Action to stay in the grid

To illustrate this, let's consider the states shown in figure 5

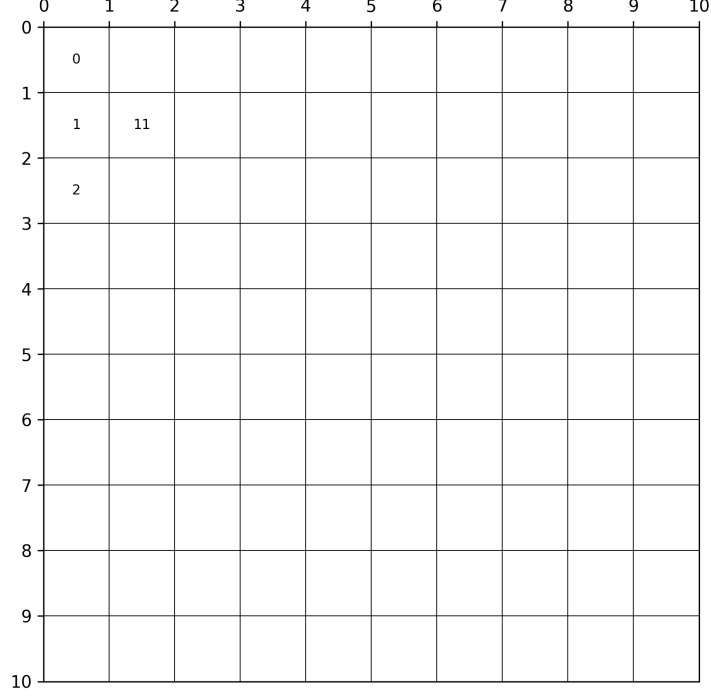


Figure 5: Edge states

The transition probabilities for taking an action to move off the grid are given below:

$$\begin{aligned}
\mathbb{P}(s_{t+1} = 0 | s_t = 1, a_t = \leftarrow) &= \frac{w}{4} \\
\mathbb{P}(s_{t+1} = 11 | s_t = 1, a_t = \leftarrow) &= \frac{w}{4} \\
\mathbb{P}(s_{t+1} = 2 | s_t = 1, a_t = \leftarrow) &= \frac{w}{4} \\
\mathbb{P}(s_{t+1} = 1 | s_t = 1, a_t = \leftarrow) &= 1 - w + \frac{w}{4}
\end{aligned}$$

The transition probabilities for taking an action to stay in the grid are given below:

$$\begin{aligned}
\mathbb{P}(s_{t+1} = 0 | s_t = 1, a_t = \uparrow) &= 1 - w + \frac{w}{4} \\
\mathbb{P}(s_{t+1} = 11 | s_t = 1, a_t = \uparrow) &= \frac{w}{4} \\
\mathbb{P}(s_{t+1} = 2 | s_t = 1, a_t = \uparrow) &= \frac{w}{4} \\
\mathbb{P}(s_{t+1} = 1 | s_t = 1, a_t = \uparrow) &= \frac{w}{4}
\end{aligned}$$

At an edge state, you can be blown off the grid in one direction. As a result, we have $\mathbb{P}(s_{t+1} = 1 | s_t = 1, a_t = \uparrow) = \frac{w}{4}$ since we can be blown off the grid in one direction and in that case we stay at the current state.

The main difference between a corner state and an edge state is that a corner state has 2 neighbors and an edge state has 3 neighbors.

2.1.4 Reward function

To simplify the project, we will assume that the reward function is independent of the current state (s) and the action that you take at the current state (a). To be specific, reward

function only depends on the state that you transition to (s'). With this simplification, we have

$$\mathcal{R}_{ss'}^a = R(s')$$

In this project, we will learn the optimal policy of an agent for two different reward functions:

- Reward function 1
- Reward function 2

The two different reward functions are displayed in figures 6 and 7 respectively

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	-10	-10	0	0	0	0
3	0	0	0	0	0	-10	-10	0	0	0	0
4	0	-10	-10	0	0	0	0	0	0	0	0
5	0	-10	-10	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0
8	0	0	-10	-10	0	0	0	0	0	0	0
9	0	0	-10	-10	0	0	0	0	0	0	1
10											

Figure 6: Reward function 1

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	-100	-100	-100	0	0	0	0
2	0	0	0	0	-100	0	-100	0	0	0	0
3	0	0	0	0	-100	0	-100	-100	-100	0	0
4	0	0	0	0	-100	0	0	0	-100	0	0
5	0	0	0	0	-100	0	0	0	-100	0	0
6	0	0	0	0	-100	0	0	0	-100	0	0
7	0	0	0	0	0	0	-100	-100	-100	0	0
8	0	0	0	0	0	0	-100	0	0	0	0
9	0	0	0	0	0	0	0	0	0	10	0

Figure 7: Reward function 2

Question 1: (10 points) For visualization purpose, generate heat maps of Reward function 1 and Reward function 2. For the heat maps, make sure you display the coloring scale. You will have 2 plots for this question

For solving question 1, you might find the following function useful:

https://matplotlib.org/api/_as_gen/matplotlib.pyplot.pcolor.html

3 Optimal policy learning using RL algorithms

In this part of the project, we will use reinforcement learning (RL) algorithm to find the optimal policy. The main steps in RL algorithm are:

- Find optimal state-value or action-value
- Use the optimal state-value or action-value to determine the deterministic optimal policy

There are a couple of RL algorithms, but we will use the Value iteration algorithm since it was discussed in detail in the lecture. We will skip the derivation of the algorithm here because it was covered in the lecture (for the derivation details please refer to the lecture slides on Reinforcement learning). We will just reproduce the algorithm below for the ease of implementation:

```

1: procedure VALUE ITERATION( $\mathcal{P}_{ss'}^a, \mathcal{R}_{ss'}^a, \mathcal{S}, \mathcal{A}, \gamma$ ):
2:   for all  $s \in \mathcal{S}$  do ▷ Initialization
3:      $V(s) \leftarrow 0$ 
4:   end for
5:    $\Delta \leftarrow \infty$ 
6:   while  $\Delta > \epsilon$  do ▷ Estimation
7:      $\Delta \leftarrow 0$ 
8:     for all  $s \in \mathcal{S}$  do
9:        $v \leftarrow V(s)$ ;
10:       $V(s) \leftarrow \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma V(s')]$ ;
11:       $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ ;
12:    end for
13:   end while
14:   for all  $s \in \mathcal{S}$  do ▷ Computation
15:      $\pi(s) \leftarrow \arg \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a [\mathcal{R}_{ss'}^a + \gamma V(s')]$ ;
16:   end for
17: end procedure  return  $\pi$ 

```

Question 2: (40 points) Create the environment of the agent using the information provided in section 2. To be specific, create the MDP by setting up the state-space, action set, transition probabilities, discount factor, and reward function. For creating the environment, use the following set of parameters:

- Number of states = 100 (state space is a 10 by 10 square grid as displayed in figure 1)
- Number of actions = 4 (set of possible actions is displayed in figure 2)
- $w = 0.1$
- Discount factor = 0.8
- Reward function 1

After you have created the environment, then write an optimal state-value function that takes as input the environment of the agent and outputs the optimal value of each state in the grid. For the optimal state-value function, you have to implement the Initialization (lines 2-4) and Estimation (lines 5-13) steps of the Value Iteration algorithm. For the estimation step, use $\epsilon = 0.01$. For visualization purpose, you should generate a figure similar to that of figure 1 but with the number of state replaced by the optimal value of that state. In this part of question, you should have 1 plot.

Let's assume that your value iteration algorithm converges in N steps. Plot snapshots of state values in 5 different steps linearly distributed from 1 to N . Report N and your step numbers. What observations do you have from the plots?

Question 3: (5 points) Generate a heat map of the optimal state values across the 2-D grid. For generating the heat map, you can use the same function provided in the hint earlier (see the hint after question 1).

Question 4: (15 points) Explain the distribution of the optimal state values across the 2-D grid. (Hint: Use the figure generated in question 3 to explain)

Question 5: (20 points) Implement the computation step of the value iteration algorithm

(lines 14-17) to compute the optimal policy of the agent navigating the 2-D state-space. For visualization purpose, you should generate a figure similar to that of figure 1 but with the number of state replaced by the optimal action at that state. The optimal actions should be displayed using arrows. Does the optimal policy of the agent match your intuition? Please provide a brief explanation. Is it possible for the agent to compute the optimal action to take at each state by observing the optimal values of it's neighboring states? In this question, you should have 1 plot.

Question 6: (10 points) Modify the environment of the agent by replacing Reward function 1 with Reward function 2. Use the optimal state-value function implemented in question 2 to compute the optimal value of each state in the grid. For visualization purpose, you should generate a figure similar to that of figure 1 but with the number of state replaced by the optimal value of that state. In this question, you should have 1 plot.

Question 7: (20 points) Generate a heat map of the optimal state values (found in question 6) across the 2-D grid. For generating the heat map, you can use the same function provided in the hint earlier.

Explain the distribution of the optimal state values across the 2-D grid. (Hint: Use the figure generated in this question to explain)

Question 8: (20 points) Implement the computation step of the value iteration algorithm (lines 14-17) to compute the optimal policy of the agent navigating the 2-D state-space. For visualization purpose, you should generate a figure similar to that of figure 1 but with the number of state replaced by the optimal action at that state. The optimal actions should be displayed using arrows. Does the optimal policy of the agent match your intuition? Please provide a brief explanation. In this question, you should have 1 plot.

Question 9:(20 points) Change the hyper parameter w to 0.6 and find the optimal policy map similar to previous question for reward functions. Explain the differences you observe. What do you think about value of new w compared to previous value? Choose the w that you think give rise to better optimal policy and use that w for the next stages of the project.

4 Inverse Reinforcement learning (IRL)

Inverse Reinforcement learning (IRL) is the task of learning an expert's reward function by observing the optimal behavior of the expert. The motivation for IRL comes from apprenticeship learning. In apprenticeship learning, the goal of the agent is to learn a policy by observing the behavior of an expert. This task can be accomplished in two ways:

1. Learn the policy directly from expert behavior
2. Learn the expert's reward function and use it to generate the optimal policy

The second way is preferred because the reward function provides a much more parsimonious description of behavior. Reward function, rather than the policy, is the most succinct, robust, and transferable definition of the task. Therefore, extracting the reward function of an expert would help design more robust agents.

In this part of the project, we will use IRL algorithm to extract the reward function.

We will use the optimal policy computed in the previous section as the expert behavior and use the algorithm to extract the reward function of the expert. Then, we will use the extracted reward function to compute the optimal policy of the agent. We will compare the optimal policy of the agent to the optimal policy of the expert and use some similarity metric between the two to measure the performance of the IRL algorithm.

4.1 IRL algorithm

For finite state spaces, there are a couple of IRL algorithms for extracting the reward function:

- Linear Programming (LP) formulation
- Maximum Entropy formulation

Since we covered the LP formulation of inverse reinforcement learning in lecture—and it is simpler than the original quadratic-program approach—we adopt that version here [1]. A step-by-step derivation can be found in our course slides [2]. The LP formulation of the IRL is given by equation 1

$$\begin{aligned}
& \underset{\mathbf{R}, t_i, u_i}{\text{maximize}} && \sum_{i=1}^{|\mathcal{S}|} (t_i - \lambda u_i) \\
& \text{subject to} && [(\mathbf{P}_{a_1}(i) - \mathbf{P}_a(i))(\mathbf{I} - \gamma \mathbf{P}_{a_1})^{-1} \mathbf{R}] \geq t_i, \quad \forall a \in \mathcal{A} \setminus a_1, \forall i \\
& && (\mathbf{P}_{a_1} - \mathbf{P}_a)(\mathbf{I} - \gamma \mathbf{P}_{a_1})^{-1} \mathbf{R} \succeq 0, \quad \forall a \in \mathcal{A} \setminus a_1 \\
& && -\mathbf{u} \preceq \mathbf{R} \preceq \mathbf{u} \\
& && |\mathbf{R}_i| \leq R_{\max}, \quad i = 1, 2, \dots, |\mathcal{S}|
\end{aligned} \tag{1}$$

In the LP given by equation 1, $\mathbf{R}$ is the reward vector ($\mathbf{R}(i) = \mathbf{R}(s_i)$), $\mathbf{P}_a$ is the transition probability matrix corresponding to the action a , λ is the adjustable penalty coefficient, and t_i 's and u_i 's are the extra optimization variables (please note that $\mathbf{u}(i) = u_i$). **Throughout IRL section we fix the discount factor to $\gamma = 0.9$.** Use the maximum absolute value of the ground-truth reward as $R_{\max}$.

For the ease of implementation, we can recast the LP in equation 1 into an equivalent form given by equation 2 using block matrices.

$$\begin{aligned}
& \underset{\mathbf{x}}{\text{maximize}} && \mathbf{c}^T \mathbf{x} \\
& \text{subject to} && \mathbf{D} \mathbf{x} \preceq \mathbf{b}, \quad \forall a \in \mathcal{A} \setminus a_1
\end{aligned} \tag{2}$$

Question 10: (10 points) Express $\mathbf{c}$, $\mathbf{x}$, $\mathbf{D}$, $\mathbf{b}$ in terms of $\mathbf{R}$, $\mathbf{P}_a$, $\mathbf{P}_{a_1}$, t_i , $\mathbf{u}$, λ and $R_{\max}$

4.2 Performance measure

In this project, we use a very simple measure to evaluate the performance of the IRL algorithm. Before we state the performance measure, let's introduce some notation:

- $O_A(s)$: Optimal action of the agent at state s
- $O_E(s)$: Optimal action of the expert at state s
-

$$m(s) = \begin{cases} 1, & O_A(s) = O_E(s) \\ 0, & \text{else} \end{cases}$$

Then with the above notation, accuracy is given by equation 3

$$Accuracy = \frac{\sum_{s \in \mathcal{S}} m(s)}{|\mathcal{S}|} \quad (3)$$

Since we are using the optimal policy found in the previous section as the expert behavior, so we will use the optimal policy found in the previous section to fill the $O_E(s)$ values. Please note that these values will be different depending on whether we used Reward Function 1 or Reward Function 2 to create the environment.

To compute $O_A(s)$, we will solve the linear program given by equation 2 to extract the reward function of the expert. For solving the linear program you can use the LP solver in python (from `cvxopt` import `solvers` and then use `solvers.lp`). Then, we will use the extracted reward function to compute the optimal policy of the agent using the value iteration algorithm you implemented in the previous section. The optimal policy of the agent found in this manner will be used to fill the $O_A(s)$ values. Please note that these values will depend on the adjustable penalty coefficient λ . We will tune λ to maximize the accuracy.

Implementation Guide

Constants $\gamma = 0.8$ $R_{\max} = \max_i |R_{\text{true}}(s_i)|$

CVXOPT options

```
from cvxopt import solvers
solvers.options.update({'abstol':1e-7,
                        'reltol':1e-6,
                        'feastol':1e-7,
                        'show_progress': False})
```

Question 11: (30 points) Sweep λ from 0 to 5 to get 500 evenly spaced values for λ . For each value of λ compute $O_A(s)$ by following the process described above. For this problem, use the optimal policy of the agent found in question 5 to fill in the $O_E(s)$ values. Then use equation 3 to compute the accuracy of the IRL algorithm for this value of λ . You need to repeat the above process for all 500 values of λ to get 500 data points. Plot λ (x -axis) against Accuracy (y -axis). In this question, you should have 1 plot.

Question 12: (5 points) Use the plot in question 11 to compute the value of λ for which accuracy is maximum. For future reference we will denote this value as $\lambda_{\max}^{(1)}$. Please report $\lambda_{\max}^{(1)}$

Question 13: (15 points) For $\lambda_{\max}^{(1)}$, generate heat maps of the ground truth reward and the extracted reward. Please note that the ground truth reward is the Reward function 1 and the extracted reward is computed by solving the linear program given by equation 2 with the λ parameter set to $\lambda_{\max}^{(1)}$. In this question, you should have 2 plots.

Question 14: (10 points) Use the extracted reward function computed in question 13, to compute the optimal values of the states in the 2-D grid. For computing the optimal values you need to use the optimal state-value function that you wrote in question 2. For visualization purpose, generate a heat map of the optimal state values across the 2-D grid (similar to the figure generated in question 3). In this question, you should have 1 plot.

Question 15: (10 points) Compare the heat maps of Question 3 and Question 14 and provide a brief explanation on their similarities and differences.

Question 16: (10 points) Use the extracted reward function found in question 13 to compute the optimal policy of the agent. For computing the optimal policy of the agent you need to use the function that you wrote in question 5. For visualization purpose, you should generate a figure similar to that of figure 1 but with the number of state replaced by the optimal action at that state. The actions should be displayed using arrows. In this question, you should have 1 plot.

Question 17: (10 points) Compare the figures of Question 5 and Question 16 and provide a brief explanation on their similarities and differences.

Question 18: (30 points) Sweep λ from 0 to 5 to get 500 evenly spaced values for λ . For each value of λ compute $O_A(s)$ by following the process described above. For this problem, use the optimal policy of the agent found in question 8 to fill in the $O_E(s)$ values. Then use equation 3 to compute the accuracy of the IRL algorithm for this value of λ . You need to repeat the above process for all 500 values of λ to get 500 data points. Plot λ (x -axis) against Accuracy (y -axis). In this question, you should have 1 plot.

Question 19: (5 points) Use the plot in question 18 to compute the value of λ for which accuracy is maximum. For future reference we will denote this value as $\lambda_{max}^{(2)}$. Please report $\lambda_{max}^{(2)}$

Question 20: (15 points) For $\lambda_{max}^{(2)}$, generate heat maps of the ground truth reward and the extracted reward. Please note that the ground truth reward is the Reward function 2 and the extracted reward is computed by solving the linear program given by equation 2 with the λ parameter set to $\lambda_{max}^{(2)}$. In this question, you should have 2 plots.

Question 21: (10 points) Use the extracted reward function computed in question 20, to compute the optimal values of the states in the 2-D grid. For computing the optimal values you need to use the optimal state-value function that you wrote in question 2. For visualization purpose, generate a heat map of the optimal state values across the 2-D grid (similar to the figure generated in question 7). In this question, you should have 1 plot.

Question 22: (10 points) Compare the heat maps of Question 7 and Question 21 and provide a brief explanation on their similarities and differences.

Question 23: (10 points) Use the extracted reward function found in question 20 to compute the optimal policy of the agent. For computing the optimal policy of the agent you need to use the function that you wrote in question 9. For visualization purpose, you should generate a figure similar to that of figure 1 but with the number of state replaced by the optimal action at that state. The actions should be displayed using arrows. In this question, you should have 1 plot.

Question 24: (10 points) Compare the figures of Question 9 and Question 23 and provide a brief explanation on their similarities and differences.

Question 25: (50 points) From the figure in question 23, you should observe that the optimal policy of the agent has two major discrepancies. Please identify and provide the causes for these two discrepancies. One of the discrepancy can be fixed easily by a slight modification to the value iteration algorithm. Perform this modification and re-run the modified value iteration algorithm to compute the optimal policy of the agent. Also, recompute the maximum accuracy after this modification. Is there a change in maximum accuracy? The second discrepancy is harder to fix and is a limitation of the simple IRL algorithm.

5 Submission

Please submit your report to Gradescope. Also submit a zip file containing your codes and report to bruinlearn. The zip file should be named as “Project3_UID1_..._UIDn.zip” where UIDx are student ID numbers of team members.

References

- [1] Andrew Y. Ng and Stuart J. Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the 17th International Conference on Machine Learning*, pages 663–670, San Francisco, CA, USA, 2000. Morgan Kaufmann.
- [2] Vwani P. Roychowdhury. Inverse reinforcement learning – lecture slides. https://bruinlearn.ucla.edu/courses/206247/files/20745735?module_item_id=7407705, 2025. EC ENGR 232E, Spring 2025.